UNIVERSITY OF HULL

Sockets and Network Programming

UNIVERSITY OF HULL

Socket Programming

- Sockets are one of the oldest (but still widely used) methods of establishing a connection across the network and exchanging data
- A socket is
  - a mechanism for creating a virtual connection between processes on the same or different machines
  - the endpoint of a communications channel

UNIVERSITY OF HULL

Sockets and Ports

- Every socket has a socket address which consists of
  - local host's network address
  - a port number
- A port is
  - a logical channel with a unique ‘port number’
  - port numbers to distinguish between different logical channels on the same computer
- Each application program or protocol has a unique port number associated with it
  - (e.g. by default HTTP uses port 80, POP uses port 110)

UNIVERSITY OF HULL

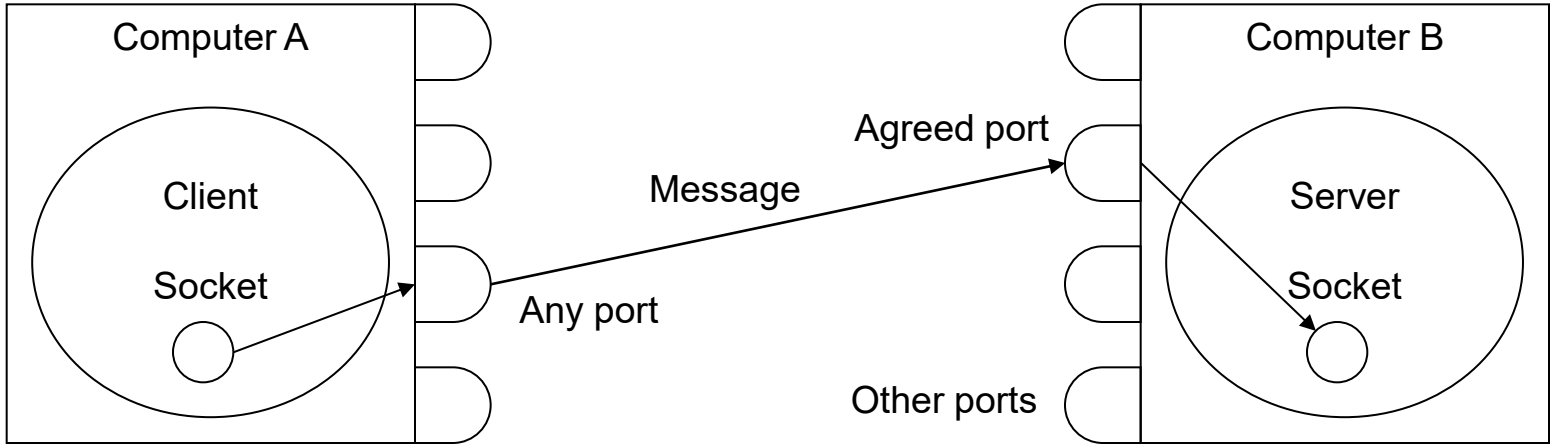
Socket API (winsock2)

- Socket
  - Protocol independent and can support a number of communication domains
    - Internet: AF_INET
    - Inter-process: AF_LOCAL
  - Types of socket
    - TCP: SOCK_STREAM
    - UDP: SOCK_DGRAM
    - IP layer: SOCK_RAW

UNIVERSITY OF HULL

Making the Connection

- A communication channel is established by logically connecting a socket on one computer (client) to a socket on another computer (server)
- Communication is a full-duplex protocol



UNIVERSITY OF HULL

Debugging

- Packet sniffers
  - Sits on any LAN workstation
  - Advantage: See all data going over the LAN
  - E.g. Microsoft Network Monitor 3.3, Wireshark (Ethereal)
- Winsock Shims
  - Sits between application and Winsock, by replicating the API
  - Disadvantage: Can only access data sent to and from the local host
  - Advantage: Can discover the sequence of calls

UNIVERSITY OF HULL

TCP

And protocols

UNIVERSITY OF HULL

Connectionless protocol

- Each packet (datagram)
  - Is an independent entity
  - Has its own address
  - Has no knowledge of other datagrams
- Analogous to sending a letter by post
- Consequences
  - Less reliable, as lost datagrams are not resent
  - Datagrams can arrive in a different order than sent

UNIVERSITY OF HULL

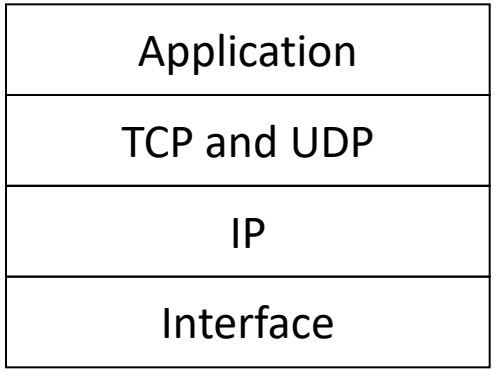
Connection-oriented protocol

- Connection-oriented protocol has three phases
  - Establish a connect between peers
  - Transfer of data
  - Withdraw the connect
- Analogous to making a phone call
  - Each packet
    - Is a dependent entity
    - Has no address of either sender or receiver
- Consequences
  - Reliable, lost packets can be resent
  - Packets can be supplied in the same order than sent

UNIVERSITY OF HULL

Connection-oriented vs connectionless protocols

- Why connectionless ?
  - Easy to support one-to-many and many-to-one communications
  - Foundation for connection-oriented protocols
    - E.g. IP is the basis of TCP



UNIVERSITY OF HULL

TCP

- TCP is connection-oriented
- Built on top of the connectionless IP protocol
- TCP packets (segments) fit within IP datagrams, but add:
  - Check sum
  - Sequence number
  - Acknowledge and retransmit mechanism
  - Knowledge of ports

UNIVERSITY OF HULL

Sample TCP Client

1. Startup the socket environment
2. Initialise a socket's data space
3. Obtain a socket
4. Establish a connection to the peer, through the socket
5. Send data to the peer
6. Receive data from the peer
7. Cleanup the socket environment

UNIVERSITY OF HULL

Simple Python TCP Client

```
1.   from socket import *

2.   port = 7500
3.   ip = '127.0.0.1'
4.   addr = (ip, port)
5.   bufferSize = 1000

6.   message = input('Enter message (def=Hello World,quit=q):').encode()
7.   if not message:
8.       message = b'Hello World'

9.   print ('Opening client')
10.  clientSocket = socket(AF_INET, SOCK_STREAM)
11.  clientSocket.connect(addr)
12.  print ('\nConnecting ', clientSocket.getsockname(), ' to ', clientSocket.getpeername())

13.  clientSocket.send(message)
14.  message = clientSocket.recv(bufferSize)
15.  print ('Receiving data ', message)

16.  print ('\nClosing client')
17.  clientSocket.close()
```

UNIVERSITY OF HULL

Simple C TCP Client

```
1.   #include <iostream>
2.   #include <ws2tcpip.h>
3.   using namespace std;

4.   int main(int, char**) {
5.       // Create version identifier
6.       WORD wVersionRequested = MAKEWORD( 2, 0 );

7.       // Startup windows sockets
8.       WSADATA wsaData;
9.       if ( WSAStartup( wVersionRequested, &wsaData )) {
10.          cerr << "Socket initialisation failed" << endl;
11.          return -1;
12.      }

13.      // Create socket data space
14.      sockaddr_in peer;
15.      peer.sin_family = AF_INET;
16.      peer.sin_port = htons(7500);
17.      inet_pton(AF_INET, "127.0.0.1", &peer.sin_addr.S_un.S_addr);
```

UNIVERSITY OF HULL

Simple C TCP Client, contd.

```
1.  // Create transfer socket
2.  char buffer;
3.  SOCKET s = socket(AF_INET, SOCK_STREAM, 0);
4.  if (s==INVALID_SOCKET) {
5.     cerr << "Create socket failed" << endl;

6.  // Connect to the peer
7.  } else if (connect(s, (sockaddr *)&peer, sizeof(peer))==SOCKET_ERROR){
8.     cerr << "Connect to peer failed with " << WSAGetLastError() << endl;

9.  // Send data
10. } else if (send(s, "1", 1, 0)==SOCKET_ERROR) {
11.    cerr << "Send failed with " << WSAGetLastError() << endl;

12. // Receive data
13. } else if (recv(s, &buffer, 1, 0)==SOCKET_ERROR) {
14.    cerr << "Receive failed with " << WSAGetLastError()  << endl;
15. } else {
16.    cout << "Message= " << buffer << endl;
17. }

18. // Cleanup windows sockets
19. WSACleanup();
20.}
```

UNIVERSITY OF HULL

Byte Ordering

- Two conflicting formats for storing integers:
  - Big Endian
    - Most significant bit in lowest memory address
    - Power PC, Spark (historical processors)
  - Little Endian
    - Least significant bit in lowest memory address
    - x86, x64
- Networking defines a neutral format
  - Network Byte Order
    - Big Endian
    - But NEVER assume it is Big Endian
- Care is also needed with floating point values

UNIVERSITY OF HULL

Server Sockets

- If we want our applications to communicate via sockets we must create
  - a client socket on the client
  - a server socket bound to a known port on the server
- A server (or listening) socket is a special type of socket which
  - Waits for and accepts incoming connections
  - Returns another socket instance which is used to send/receive data

UNIVERSITY OF HULL

Sample TCP Server

1. Startup the socket environment
2. Initialise a socket's data space
3. Obtain a socket
4. Bind the port and address to the socket
5. Mark the socket as a listening socket
6. Block until a new connection is ready, then accept it and return a socket for the new connection
7. Receive data from the peer
8. Send data to the peer
9. Cleanup the socket environment

UNIVERSITY OF HULL

Simple Python TCP Server

```
1.  from socket import *

2.  port = 7500
3.  ip = '' #INADDR_ANY
4.  addr = (ip, port)
5.  bufferSize = 1000
6.  message = ''

7.  print ('Opening echo server')
8.  listeningSocket = socket(AF_INET, SOCK_STREAM)
9.  listeningSocket.setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)
10. listeningSocket.bind(addr)
11. listeningSocket.listen(5)

12. while (message != b'q'):
13.     print ('\nListening on ', addr)
14.     dataSocket, peerAddr = listeningSocket.accept()
15.     print ('Connecting ', dataSocket.getsockname(), ' to ', dataSocket.getpeername())

16.     message = dataSocket.recv(bufferSize)
17.     print ('Echoing message ', message)
18.     dataSocket.send(message)
19.     dataSocket.close()

20. print ('\nClosing echo server')
21. listeningSocket.close()
```

UNIVERSITY OF HULL

Simple C TCP Server

```
1.   #include <iostream>
2.   #include <ws2tcpip.h>
3.   using namespace std;

4.   int main(int, char**) {
5.     // Create version identifier
6.     WORD wVersionRequested = MAKEWORD( 2, 0 );

7.     // Startup windows sockets
8.     WSADATA wsaData;
9.     if ( WSAStartup( wVersionRequested, &wsaData )) {
10.      cerr << "Socket initialisation failed" << endl;
11.      return -1;
12.    }

13.    // Create socket data space
14.    sockaddr_in peer;
15.    peer.sin_family = AF_INET;
16.    peer.sin_port = htons(7500);
17.    inet_pton(peer.sin_family, htonl( INADDR_ANY ), &peer.sin_addr);
```

UNIVERSITY OF HULL

Simple C TCP Server, contd.

```
1.   // Create listening socket
2.   SOCKET s = socket(AF_INET, SOCK_STREAM, 0);
3.   if (s==INVALID_SOCKET) {
4.      cerr << "Create socket failed" << endl;
5.   } else if (bind(s, (sockaddr *)&peer, sizeof(peer))==SOCKET_ERROR) {
6.      cerr << "Bind failed with " << WSAGetLastError()  << endl;
7.   } else if (listen(s, 5)==SOCKET_ERROR) {
8.      cerr << "Listen failed with " << WSAGetLastError()  << endl;
9.   } else {
```

UNIVERSITY OF HULL

Simple C TCP Server, contd.

```
1.      // Create transfer socket
2.      char buffer;
3.      while (buffer != 'q') {
4.         SOCKET s1 = accept(s, NULL, NULL);
5.         if (s1==INVALID_SOCKET) {
6.            cerr << "Accept failed with " << WSAGetLastError() << endl;
7.            // Receive data
8.         } else if (recv(s1, &buffer, 1, 0)==SOCKET_ERROR) {
9.            cerr << "Receive failed with " << WSAGetLastError() << endl;
10.        } else {
11.           cout << "Message= " << buffer << endl;
12.           // Send data
13.           if (send(s1, "2", 1, 0)==SOCKET_ERROR) {
14.              cerr << "Send failed with " << WSAGetLastError() << endl;
15.           }
16.        }
17.        closesocket(s1);
18.     }
19.  }
20.  // Delay
21.  cout << "Waiting..." << endl; char ch; cin >> ch;
22.  // Cleanup windows sockets
23.  WSACleanup();
24.  }
```

UNIVERSITY OF HULL

Closing connections

- TCP is full-duplex
- Can close half of the connection, and still transmit data on the other half
- TCP connection terminates when both halves are closed
- Normal protocol
  - Sender closes half of the connection
  - Receiver detects closure
  - Receiver closes half of the connection
  - Sender detects closure

UNIVERSITY OF HULL

Closing a TCP socket

- Closing sequence
  - Call shutdown() with parameters SD_RECEIVE, SD_SEND or SD_BOTH
    - `shutdown (s, SD_SEND);`
  - Call recv until zero returned, or SOCKET_ERROR.
    - `do {`
    - `    iResult = recv(s, recvbuf, recvbuflen, 0);`
    - `while( iResult > 0 );`
  - Call closesocket()
    - `closesocket (s);`

UNIVERSITY OF HULL

Detecting a socket error

- Reading with error handling

```
1.  const int bytesRead = recv(handle(), buffer, size, flags);
2.     if (bytesRead == SOCKET_ERROR) {
3.        // Other receiving error
4.        return SOCKET_ERROR;
5.     }
6.     if (bytesRead == 0) {
7.        // Connection closed by peer
8.         return SOCKET_ERROR; // connection closure
9.     }
10.    return bytesRead;
```

UNIVERSITY OF HULL

Detecting a socket error, with timeout

- Reading with error handling

```
1.  const int bytesRead = recv(handle(), buffer, size, flags);
2.     if (bytesRead == SOCKET_ERROR) {
3.        const auto error = WSAGetLastError();
4.        if (error == WSAETIMEDOUT) {
5.           // Timeout occurred
6.           return 0; // Indicate timeout
7.        }
8.        else {
9.           // Other receiving error
10.          return SOCKET_ERROR;
11.       }
12.    }
13.    if (bytesRead == 0) {
14.       // Connection closed by peer
15.        return SOCKET_ERROR; // connection closure
16.    }
17.    return bytesRead;
```

UNIVERSITY OF HULL

Detecting network failures

- Options
  - Do nothing
  - Use a command/response structure
    - Sender waits to get a response to confirm data has been sent correctly.
  - Send “are you there” messages
    - Sender periodically sends small “are you there” messages and waits for a reply
  - Use ICMP ping
  - TCP keep alive feature
    - Kills local connection if remote peer is no longer available

UNIVERSITY OF HULL

TCP Acknowledge and retransmit mechanism: receiver

- Each TCP connection maintains a receive window
  - First acceptable byte
  - Last acceptable byte
- Bytes arriving before the first acceptable byte are usually ignored, as they are resends
- Bytes arriving after the last acceptable byte are usually queued, as they would otherwise cause a buffer overflow, or discarded
- Bytes arriving which include the first acceptable byte are made available to the application or queued. Also
  - The receive window is adjusted
  - An acknowledge message (ACK) is sent to the peer

UNIVERSITY OF HULL

TCP Acknowledge and retransmit mechanism: sender

- Each TCP connection maintains a send window
  - Bytes sent but not acknowledged
  - Bytes to be sent
- Bytes are sent from the “to be sent” window
  - Bytes are transferred from the “to be sent” window to the “to be acknowledged” window
  - Retransmission timeout (RTO) is started
- A RTO timeout causes the bytes to be resent
- What causes a RTO timeout?
- What happens at the receiver if duplicate bytes are received?

UNIVERSITY OF HULL

UDP

UNIVERSITY OF HULL

UDP

- UDP is connectionless
- Built on top of the connectionless IP protocol
- UDP packets fit within IP datagrams, but add:
  - Check sum
  - Notion of ports
- Ports allow de-multiplexing of packets to a single IP address
  - A combination of address and port is used to associate a packet with a socket
  - Ports are available in both UDP and TCP

UNIVERSITY OF HULL

Sample UDP Client

1. Startup the socket environment
2. Initialise a socket's data space
3. Obtain a socket
4. Send data to the peer
5. Receive data from the peer
6. Cleanup the socket environment

UNIVERSITY OF HULL

Simple Python UDP Client

```
1.  from socket import *

2.  port = 4300
3.  ip = '127.0.0.1'
4.  addr = (ip, port)
5.  bufferSize = 1000

6.  message = input('Enter message (default=Hello World, quit=q):').encode()
7.  if not message:
8.      message = b'Hello World'

9.  print ('Opening client')
10. clientSocket = socket(AF_INET, SOCK_DGRAM)

11. clientSocket.sendto(message, addr)
12. message, peerAddr = clientSocket.recvfrom(bufferSize)
13. print ('Receiving data ', message, ' from ', peerAddr)

14. print ('Closing client')
15. clientSocket.close()
```

UNIVERSITY OF HULL

Simple C UDP Client

```
1.   #include <iostream>
2.   #include <ws2tcpip.h>
3.   using namespace std;

4.   int main(int, char**) {
5.       // Create version identifier
6.       WORD wVersionRequested = MAKEWORD( 2, 0 );

7.       // Startup windows sockets
8.       WSADATA wsaData;
9.       if ( WSAStartup( wVersionRequested, &wsaData )) {
10.          cerr << "Socket initialisation failed" << endl;
11.          return -1;
12.      }

13.      // Create socket data space
14.      sockaddr_in peer;
15.      peer.sin_family = AF_INET;
16.      peer.sin_port = htons(7500);
17.      inet_pton(peer.sin_family, "127.0.0.1", &peer.sin_addr);
```

UNIVERSITY OF HULL

Simple C UDP Client, contd.

```
1.    // Create transfer socket
2.    char buffer;
3.    SOCKET s = socket(AF_INET, SOCK_DGRAM, 0);
4.    if (s==INVALID_SOCKET) {
5.       cerr << "Create socket failed" << endl;
6.
7.    // Send data
8.    } else if (sendto(s, "1", 1, 0, (sock_addr*)&peer, sizeof(peer)) == SOCKET_ERROR) {
9.       cerr << "Send failed with " << WSAGetLastError() << endl;

10.   // Receive data
11.   } else if (recvfrom(s, &buffer, 1, 0, 0, 0) == SOCKET_ERROR) {
12.      cerr << "Receive failed with " << WSAGetLastError() << endl;

13.   } else {
14.      cout << "Message= " << buffer << endl;
15.   }

16.   // Cleanup windows sockets
17.   WSACleanup();
18.  }
```

UNIVERSITY OF HULL

Sample UDP Server

1. Startup the socket environment
2. Initialise a socket's data space
3. Obtain a socket
4. Bind the port and address to the socket
5. Receive data and addr from the peer
6. Send data to the peer
7. Cleanup the socket environment

UNIVERSITY OF HULL

Simple Python UDP Server

```
1.  from socket import *

2.  port = 4300
3.  ip = '' #INADDR_ANY
4.  addr = (ip, port)
5.  bufferSize = 1000
6.  message = ''

7.  print ('Opening echo server')
8.  dataSocket = socket(AF_INET, SOCK_DGRAM)
9.  dataSocket.setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)
10. dataSocket.bind(addr)

11. while (message != b'q'):
12.     message, peerAddr = dataSocket.recvfrom(bufferSize)
13.     print ('\nEchoing message ', message, ' to ', peerAddr)
14.     dataSocket.sendto(message, peerAddr)

15. print ('\nClosing echo server')
16. dataSocket.close()
```

UNIVERSITY OF HULL

Simple C UDP Server

```
1.  #include <iostream>
2.  #include <WS2tcpip.h>
3.  using namespace std;

4.  int main(int, char**) {
5.      // Create version identifier
6.      WORD wVersionRequested = MAKEWORD( 2, 0 );

7.      // Startup windows sockets
8.      WSADATA wsaData;
9.      if ( WSAStartup( wVersionRequested, &wsaData )) {
10.             cerr << "Socket initialisation failed" << endl;
11.             return -1;
12.     }

13.     // Create socket data space
14.     sockaddr_in peer;
15.     peer.sin_family = AF_INET;
16.     peer.sin_port = htons(7500);
17.     peer.sin_addr.S_un.S_addr = htonl( INADDR_ANY );
```

UNIVERSITY OF HULL

Simple C UDP Server, contd.

```
1.  // Create listening socket
2.  SOCKET s = socket(AF_INET, SOCK_DGRAM, 0);
3.  if (s==INVALID_SOCKET) {
4.     cerr << "Create socket failed" << endl;
5.  } else if (bind(s, (sockaddr *)&peer, sizeof(peer))==SOCKET_ERROR) {
6.     cerr << "Bind failed with " << WSAGetLastError()  << endl;
7.
```

UNIVERSITY OF HULL

Simple C UDP Server, contd.

```
1.  } else {
2.      sockaddr_in clientAddr;
3.      int addrSize = sizeof(clientAddr);
4.      char buffer = ' ';
5.      while (buffer != 'q') {

6.          // Receive data
7.          if (recvfrom(s, &buffer, 1, 0, (sock_addr*)&clientAddr, &addrSize)==SOCKET_ERROR) {
8.              cerr << "Receive failed with " << WSAGetLastError() << endl;

9.          // Send data
10.         } else if (sendto(s, "2", 1, 0, (sock_addr*)&clientAddr, sizeof(clientAddr))==SOCKET_ERROR) {
11.             cerr << "Send failed with " << WSAGetLastError() << endl;

12.         } else {
13.             cout << "Message= " << buffer << endl;
14.         }
15.     }
16. }
17. // Cleanup windows sockets
18. WSACleanup();
19. }
```